Politechnika Śląska

UCZELNIA BADAWCZA
INICJATYWA DOSKONAŁOŚCI

Sprawozdanie

Rodzaj zajęć
Laboratorium

Przedmiot
Systemy mikroprocesorowe i wbudowane

Rok akademicki	Miasto	Kierunek	Semestr	Prowadzący	Grupa	Sekcja
2025/2026	G	INF	5	JP	5	10

Planowany termin wykonywania ćwiczenia		Faktyczny termin wykonania ćwiczenia	
Data	Godzina	Data	Godzina
12.01.2025	9:45		

Numer ćwiczenia	Temat ćwiczenia
23	Mikrokontrolery AVR cz.3 – Liczniki&Przerwania

Skład sekcji		
	Imię i nazwisko	Uwagi
1	Piotr Ptasznik	
2		
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		
13		
14		
15		

1.Treść zadania:

Na początku wyłącz wszystkie diody. Następnie czekaj na naciśnięcie dowolnego przycisku na porcie A. Po naciśnięciu przycisku skopiuj kolejne bajty z tablicy TAB_ROM do tablicy TAB_RAM w sposób sekwencyjny. Jeżeli TAB_ROM jest większa niż TAB_RAM, kopiowanie ma zawijać się do początku TAB_RAM po osiągnięciu jej końca, aż do odczytania całej TAB_ROM. Każdy skopiowany bajt wyświetl na diodach portu C, wykonując przerwę około 1000 ms między kolejnymi wartościami. Po zakończeniu włącz diodę LED na porcie B.6: 1000 ms – ON, następnie 250 ms – OFF, potem 500 ms – ON, a na końcu przełącz na OFF.

2.Rozwiązanie:

Zadanie zrealizowano przy użyciu następującego kodu:

```
////////////////////////////////////////////////////////////////////////////////
// Laboratory AVR Microcontrollers Part3
//
//
// Author: Piotr Ptasznik
//
// Group: 5
// Section: 10
// Task: 8
//
//Initially, turn off all diodes. Wait for a button press, then copy tables when any button on port A is
//pressed. Write a program that copies each byte from TAB_ROM to TAB_RAM in sequence. If
//TAB_ROM is larger than TAB_RAM, the copying wraps around, so bytes from the start of TAB_RAM
//are copied again after reaching its end, until all of TAB_ROM is read. Display each copied byte on
//port C diodes, pausing about1000ms. between each. Afterwards, turn on the LED
//connected to port B.6. After that: Turn LED connected to Port B.6 exactly: for 1000ms - ON, next
//250ms - OFF, next for 500ms - ON and then switch to OFF.
//
//
////////////////////////////////////////////////////////////////////////////////
.nolist
.include "m2560def.inc"
.list

; -------------------------
; Constants
; -------------------------
.equ xlength = 2

; Timer1 tick = 10ms:
; f_timer = 16MHz/256 = 62500 Hz
; counts for 10ms = 625
; OCR1A = 625-1 = 624
.equ OCR1A_10MS = 624

.equ TICKS_250MS  = 25
.equ TICKS_500MS  = 50
.equ TICKS_1000MS = 100

; -------------------------
; SRAM
; -------------------------
.DSEG
.ORG 0x200
TAB_RAM:   .BYTE xlength
t1_flag:   .BYTE 1

; -------------------------
; CODE
```

```
; -------------------------
.CSEG
.org 0x0000
jmp RESET

; Interrupt vectors (unused -> RETI), Timer1 COMPA used
jmp EXT_INT0
jmp EXT_INT1
jmp EXT_INT2
jmp EXT_INT3
jmp EXT_INT4
jmp EXT_INT5
jmp EXT_INT6
jmp EXT_INT7
jmp HPCINT0
jmp HPCINT1
jmp HPCINT2
jmp WDT
jmp TIM2_COMPA
jmp TIM2_COMPB
jmp TIM2_OVF
jmp TIM1_CAPT
jmp TIM1_C0MPA_HANDLER
jmp TIM1_C0MPB
jmp TIM1_COMPC
jmp TIM1_0VF
jmp TIM0_COMPA
jmp TIM0_COMPB
jmp TIM0_OVF
jmp SPI_STC
jmp USART0_RXC
jmp USART0_UDRE
jmp USART0_TXC
jmp ANA_COMP
jmp HADC
jmp EE_RDY
jmp TIM3_CAPT
jmp TIM3_COMPA
jmp TIM3_COMPB
jmp TIM3_COMPC
jmp TIM3_OVF
jmp USART1_RXC
jmp USART1_UDRE
jmp USART1_TXC
jmp TWI
jmp SPM_RDY
jmp TIM4_CAPT
jmp TIM4_COMPA
jmp TIM4_COMPB
jmp TIM4_COMPC
jmp TIM4_OVF
jmp TIM5_CAPT
jmp TIM5_COMPA
jmp TIM5_COMPB
jmp TIM5_COMPC
jmp TIM5_OVF
jmp USART2_RXC
jmp USART2_UDRE
jmp USART2_TXC
jmp USART3_RXC
jmp USART3_UDRE
jmp USART3_TXC

; ---------------------------------------------------------
; Unused ISRs
; ---------------------------------------------------------
```

```
EXT_INT0:
EXT_INT1:
EXT_INT2:
EXT_INT3:
EXT_INT4:
EXT_INT5:
EXT_INT6:
EXT_INT7:
HPCINT0:
HPCINT1:
HPCINT2:
WDT:
TIM2_COMPA:
TIM2_COMPB:
TIM2_OVF:
TIM1_CAPT:
TIM1_C0MPB:
TIM1_COMPC:
TIM1_0VF:
TIM0_COMPA:
TIM0_COMPB:
TIM0_OVF:
SPI_STC:
USART0_RXC:
USART0_UDRE:
USART0_TXC:
ANA_COMP:
HADC:
EE_RDY:
TIM3_CAPT:
TIM3_COMPA:
TIM3_COMPB:
TIM3_COMPC:
TIM3_OVF:
USART1_RXC:
USART1_UDRE:
USART1_TXC:
TWI:
SPM_RDY:
TIM4_CAPT:
TIM4_COMPA:
TIM4_COMPB:
TIM4_COMPC:
TIM4_OVF:
TIM5_CAPT:
TIM5_COMPA:
TIM5_COMPB:
TIM5_COMPC:
TIM5_OVF:
USART2_RXC:
USART2_UDRE:
USART2_TXC:
USART3_RXC:
USART3_UDRE:
USART3_TXC:
    reti

; -------------------------
; Register aliases
; -------------------------
.def rTmp    = r16
.def rData   = r17
.def rOuter  = r18
.def rMid    = r19
.def rInner  = r20
.def rBuf    = r21
```

```
.def rHasBuf = r22
.def rTicks  = r23

; ---------------------------------------------------------
; Timer1 COMPA ISR: set flag only
; ---------------------------------------------------------
TIM1_C0MPA_HANDLER:
   push rTmp
   in   rTmp, SREG
   push rTmp

   ldi  rTmp, 1
   sts  t1_flag, rTmp

   pop  rTmp
   out  SREG, rTmp
   pop  rTmp
   reti

; ---------------------------------------------------------
; RESET
; ---------------------------------------------------------
RESET:
   cli

   ldi  rTmp, HIGH(RAMEND)
   out  SPH, rTmp
   ldi  rTmp, LOW(RAMEND)
   out  SPL, rTmp

   clr  rHasBuf
; ---------------------------------------------------------
; Ports init (A..L)
; ---------------------------------------------------------
   ; PORTC: LEDs output, OFF (active-high when SV1=2-1)
   ldi  rTmp, 0xFF
   out  DDRC, rTmp
   ldi  rTmp, 0x00
   out  PORTC, rTmp

   ; PORTB: init ALL 3 LEDs as outputs, OFF
   ldi  rTmp, (1<<5)|(1<<6)|(1<<7)
   out  DDRB, rTmp
   ldi  rTmp, 0x00
   out  PORTB, rTmp

   ; PORTA: inputs; with SV2=2-3 external pull-ups -> internal pull-ups OFF
   ldi  rTmp, 0x00
   out  DDRA, rTmp
   out  PORTA, rTmp

   ; Remaining ports: inputs, no pullups
   ldi  rTmp, 0x00
   out  DDRD,  rTmp
   out  PORTD, rTmp

   sts  DDRE,  rTmp
   sts  PORTE, rTmp

   sts  DDRF,  rTmp
   sts  PORTF, rTmp
   sts  DDRG,  rTmp
   sts  PORTG, rTmp
   sts  DDRH,  rTmp
   sts  PORTH, rTmp
   sts  DDRJ,  rTmp
```

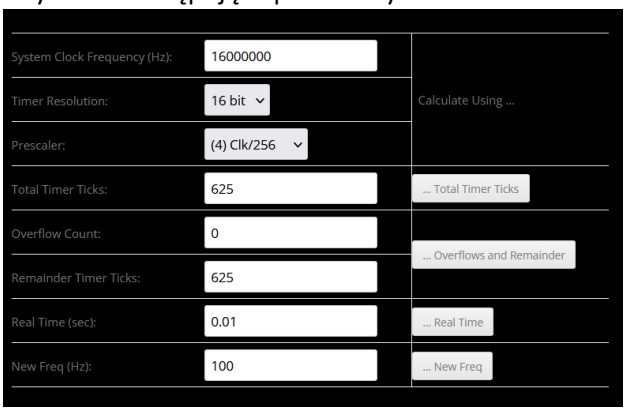
```
    sts  PORTJ, rTmp
    sts  DDRK,  rTmp
    sts  PORTK, rTmp
    sts  DDRL,  rTmp
    sts  PORTL, rTmp

; ---------------------------------------------------------
; Timer1 setup: CTC mode, OCR1A match every 10ms
; ---------------------------------------------------------
    ldi  rTmp, 0
    sts  t1_flag, rTmp

    clr  rTmp
    sts  TCCR1A, rTmp
    ldi  rTmp, (1<<WGM12) | (1<<CS12)   ; CTC + prescaler /256
    sts  TCCR1B, rTmp

    ldi  rTmp, HIGH(OCR1A_10MS)
    sts  OCR1AH, rTmp
    ldi  rTmp, LOW(OCR1A_10MS)
    sts  OCR1AL, rTmp

    ldi  rTmp, 0
    sts  TCNT1H, rTmp
    sts  TCNT1L, rTmp

    ldi  rTmp, (1<<OCF1A)
    sts  TIFR1, rTmp

    ldi  rTmp, (1<<OCIE1A)
    sts  TIMSK1, rTmp

    sei

; ---------------------------------------------------------
; Wait for button press on PORTA (active-low)
; Pressed if PINA != 0xFF
; ---------------------------------------------------------
WaitButton:
    in   rTmp, PINA
    cpi  rTmp, 0xFF
    breq WaitButton

    in   rTmp, PINA
    cpi  rTmp, 0xFF
    breq WaitButton

; ---------------------------------------------------------
; Init pointers:
; X -> TAB_RAM
; Y -> TAB_RAM end (wrap)
; Z:RAMPZ -> TAB_ROM (byte addr) for ELPM, placed >64kB
; ---------------------------------------------------------
    clr  rHasBuf
                clr  rData
    clr  rBuf
    ldi  XL, low(TAB_RAM)
    ldi  XH, high(TAB_RAM)

    ldi  YL, low(TAB_RAM + xlength)
    ldi  YH, high(TAB_RAM + xlength)

    ldi  ZL, LOW(TAB_ROM*2)
    ldi  ZH, HIGH(TAB_ROM*2)
    ldi  rTmp, BYTE3(TAB_ROM*2)
    out  RAMPZ, rTmp
```

```
; ------------------------------------------------------------
; Copy loop: read each byte from FLASH using ELPM, show on PORTC,
; delay ~1000ms, wrap RAM, stop on sentinel 0x00,0x00
; ------------------------------------------------------------
CopyLoop:
   tst  rHasBuf
   breq ReadFromFlash
   mov  rData, rBuf
   clr  rHasBuf
   rjmp ProcessByte

ReadFromFlash:
   elpm rData, Z+
   tst  rData
   brne ProcessByte

   ; if rData==0, peek next
   elpm rTmp, Z+
   tst  rTmp
   breq CopyDone

   mov  rBuf, rTmp
   ldi  rHasBuf, 1

ProcessByte:
   st   X+, rData
   out  PORTC, rData
               ldi  rTicks, TICKS_1000MS
               rcall DelayTicks_T1
               ldi  rTmp, 0x00
               out  PORTC, rTmp


   ; wrap RAM if X==Y
   cp   XL, YL
   cpc  XH, YH
   brne CopyLoop

   ldi  XL, low(TAB_RAM)
   ldi  XH, high(TAB_RAM)
   rjmp CopyLoop

; ------------------------------------------------------------
; After copy: PB6 timing
; ON 1000ms, OFF 250ms, ON 500ms, then OFF
; ------------------------------------------------------------
CopyDone:
   ldi  rTmp, 0x00
   out  PORTC, rTmp

   sbi  PORTB, 6
   ldi  rTicks, TICKS_1000MS
   rcall DelayTicks_T1

   cbi  PORTB, 6
   ldi  rTicks, TICKS_250MS
   rcall DelayTicks_T1

   sbi  PORTB, 6
   ldi  rTicks, TICKS_500MS
   rcall DelayTicks_T1

   cbi  PORTB, 6

End:
```

```
    rjmp End

; ------------------------------------------------------------
; DelayTicks_T1: waits rTicks * 10ms using t1_flag
; ------------------------------------------------------------
DelayTicks_T1:
DelayTicks_Loop:
    lds  rTmp, t1_flag
    tst  rTmp
    breq DelayTicks_Loop

    ldi  rTmp, 0
    sts  t1_flag, rTmp

    dec  rTicks
    brne DelayTicks_Loop
    ret

; ------------------------------------------------------------
; Optional debounce ~20ms
; ------------------------------------------------------------
Delay20ms_SW:
    push rOuter
    push rMid
    push rInner

    ldi  rOuter, 7
D20_O:
    ldi  rMid, 126
D20_M:
    ldi  rInner, 120
D20_I:
    dec  rInner
    brne D20_I
    dec  rMid
    brne D20_M
    dec  rOuter
    brne D20_O

    pop  rInner
    pop  rMid
    pop  rOuter
    ret

; ------------------------------------------------------------
; TAB_ROM in FLASH, forced above 64kB for far addressing
; (AVR .org in .cseg is WORD address)
; ------------------------------------------------------------
.org 0x11000

TAB_ROM:
    .db  0x01
    .db  0x00, 0x00

.EXIT
```

Timer został wyznaczony przy pomocy kalkulatora ze strony: https://eleccelerator.com/avr-timer-calculator/

Uzyskano następujące parametry dla real time = 0.01s:

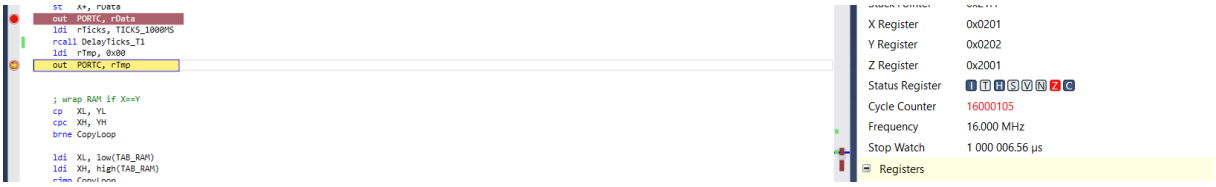


2.Testy

Delay podczas kopiowania **1000ms**

Przed:

Po:

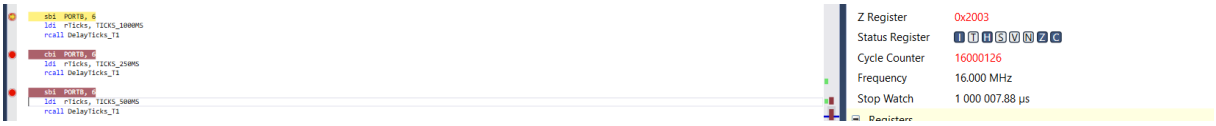


Różnica:

1000006.56 µs -6.50 µs= 1000000,06 µs

Oczekiwany wynik jest niemal idealny

Delay po zakończeniu kopiowania **1000ms**, przed



po

Różnica:

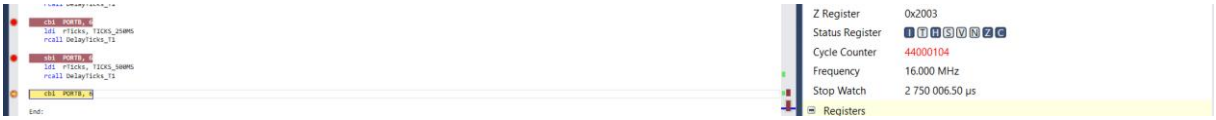
2000006.38 µs -1000007.88 µs =999998,5 – różnica 1,5 µs od założeń

250ms

Różnica:

2250006.44 µs – 2000006.38 µs= 250000,06 µs wynik niemal idealny

500ms



Różnica:

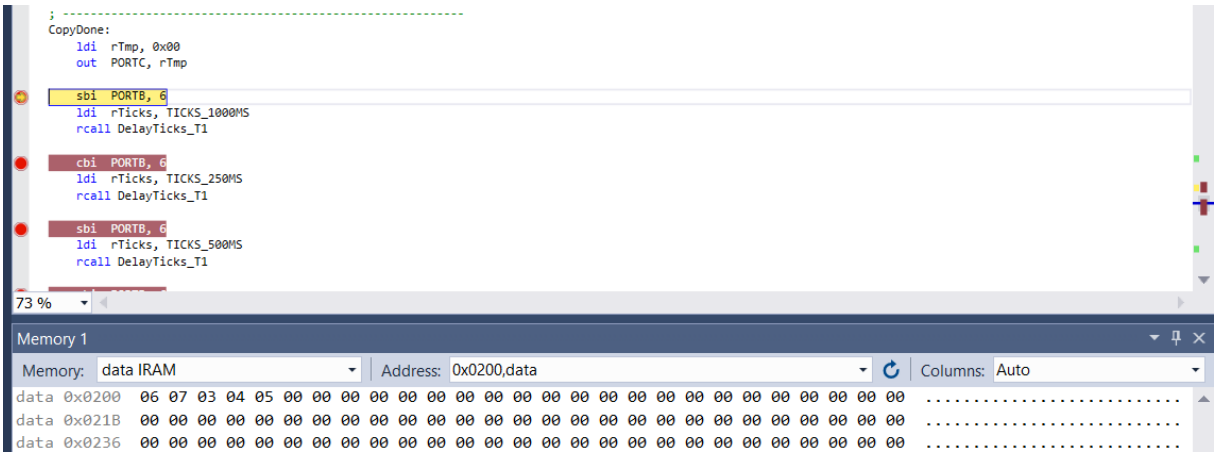
2750006.5 µs – 2250006.44 µs = 500000.06 µs ponownie niemal idealny wynik

Test poprawnej logiki działania programu.

Program uruchomiono dla długości TAB_RAM = 5 oraz takiej zawartości TAB_ROM:

```
TAB_ROM:
  .db  0x01, 0x02, 0x03, 0x04, 0x05, 0x06, 0x07
  .db  0x00, 0x00
```

Po zakończeniu kopiowania stan TAB_RAM wygląda następująco



Co wskazuje na poprawne zapętlanie tablicy zgodnie z założeniem zadania

3.Wnioski

Podczas realizacji ćwiczenia zaimplementowano funkcjonalność obsługi liczników i przerwań w mikrokontrolerach AVR, analogicznie jak w poprzednich zadaniach. Program działa poprawnie, kopiując dane z pamięci ROM do RAM nadpisując TAB_RAM gdy TAM_ROM jest większa. Przeprowadzone testy potwierdziły skuteczność opóźnień, które osiągnęły dokładność w granicach ±5µs, zgodnie z założeniami laboratorium. Implementacja została sprawdzona pod kątem poprawności i spełnia oczekiwania wynikające z założeń teoretycznych.